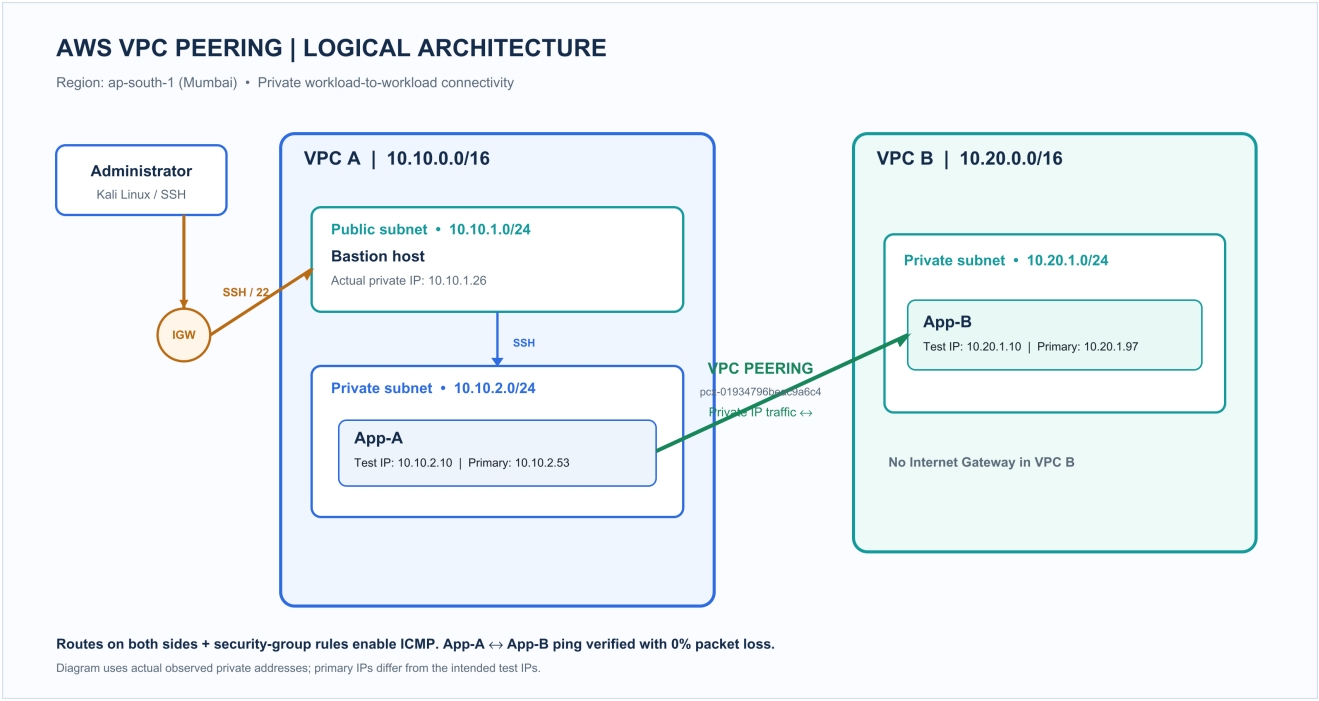
PROJECT REPORT

AWS VPC Peering

Private connectivity between two Amazon Virtual Private Clouds using route tables, security groups and a bastion host.

PREPARED BY	PROJECT DETAILS
Stany Joseph Batch: [Enter batch name / number]	**Cloud:** Amazon Web Services (AWS) **Region:** Asia Pacific (Mumbai), `ap-south-1` **Completion date:** [Enter actual completion date]



Submission note: replace bracketed fields, add the named evidence screenshots, and verify pending tests/cleanup before final submission.

1. Executive Summary

This project establishes private IPv4 communication between two VPCs in the Mumbai AWS Region. VPC A hosts a bastion server and App-A; VPC B hosts App-B. A VPC peering connection provides a direct private network path between the VPC CIDR ranges. Route tables define the path in each direction, while security groups restrict which traffic is permitted.

The two directional ICMP tests supplied for this report succeeded with zero packet loss when the intended secondary private IP addresses were explicitly selected as the source and destination. SSH-through-bastion, controlled failure/recovery, and cleanup evidence must be added only after those activities are verified.

2. Introduction

Amazon VPC Peering connects two VPCs so resources can communicate using private IP addresses, provided that the VPC CIDRs do not overlap, the peering connection is active, both sides have the necessary routes, and security controls permit the traffic. Peering does not automatically create routes or grant access; those settings must be configured separately.

3. Project Objectives

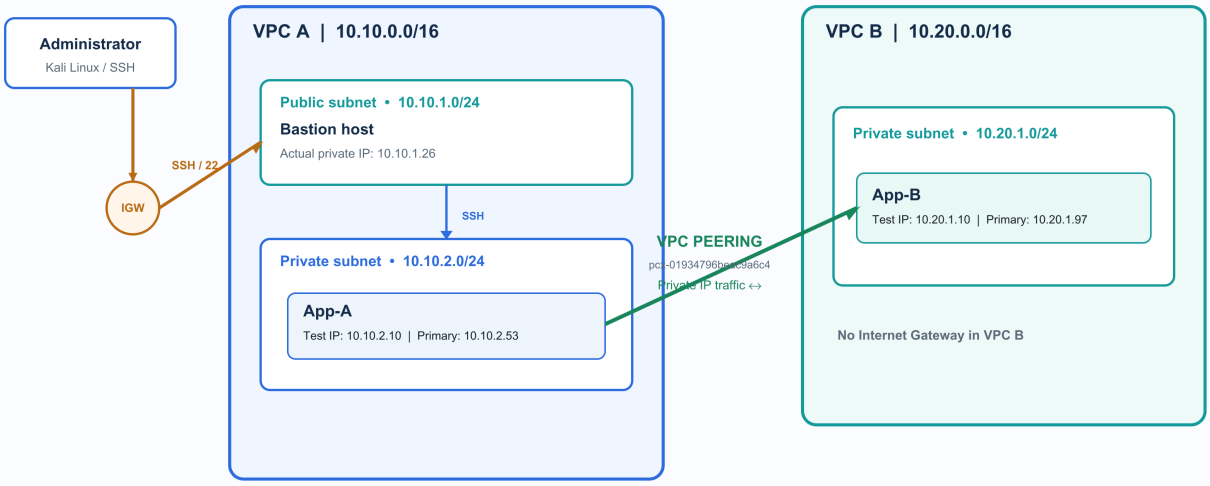
Objective	Evidence / expected outcome
Network design	Two non-overlapping VPC CIDRs and their subnets are documented.
Routing	Each relevant route table contains the peer CIDR route and correct subnet association.
Secure administration	Bastion provides the administration hop; application servers have no public IPv4 address.
Connectivity	Forward and reverse ICMP tests show successful private connectivity.
Troubleshooting	A controlled ICMP failure and recovery are captured and explained.
Responsible completion	Screenshots are saved before lab resources are cleaned up.

4. Architecture

The logical architecture is shown on the cover. The bastion is placed in VPC A's public subnet. App-A and App-B are placed in private subnets in separate VPCs. The peering connection joins the VPC networks; it is not an internet gateway and does not provide VPC B with internet access.

AWS VPC PEERING | LOGICAL ARCHITECTURE

Region: ap-south-1 (Mumbai) • Private workload-to-workload connectivity



Routes on both sides + security-group rules enable ICMP. App-A ↔ App-B ping verified with 0% packet loss.

Diagram uses actual observed private addresses; primary IPs differ from the intended test IPs.

5. Resource Inventory

The following IDs and addresses are based on the environment details recorded during troubleshooting. Confirm each ID in the AWS Console before submitting; enter values marked 'not recorded' from your actual account.

Resource	Name / description	Recorded ID / value
VPC A	ewb-vpc-a	vpc-08092d12e8b63a296
VPC B	ewb-vpc-b	vpc-0cabf8113fbdd7d46
Peering	ewb-a-b-peer	pcx-01934796beac9a6c4
App-A EC2	ewb-app-a	i-0c275e8611a22903a
App-A ENI	Primary network interface	eni-0a7048efe76a299c5
App-B EC2	ewb-app-b	i-02eb907d7b30eaac6
App-B ENI	Primary network interface	eni-0427654500ae809fc
Bastion	ewb-bastion	Instance ID: not recorded here
Public subnet A	ewb-a-public	Subnet ID: not recorded here
Private subnet A	ewb-a-private	subnet-0558eab73a3d67ba1
Private subnet B	ewb-b-private	subnet-0b1529b9710d77133
Public route table	ewb-rt-a-public	rtb-01b8e85216ac7c4d2
Private route table A	ewb-rt-a-private	rtb-07be1132cea4a051d
Private route table B	ewb-rt-b-private	rtb-0e06e4260a368e14d
App-B security group	ewb-sg-app-b	sg-09d1174a8b11ca1d9
App-A security group	ewb-sg-app-a	ID: not recorded here
Bastion security group	ewb-sg-bastion	ID: not recorded here
Internet gateway	ewb-igw-a	ID: igw-030431e12118d6dad
Region	Mumbai	ap-south-1

Address plan and observed addresses

Component	Subnet CIDR	Observed address details
VPC A	10.10.0.0/16	Local route retained
Public subnet A	10.10.1.0/24	Bastion actual private IP: 10.10.1.26; public IP previously observed: 13.207.38.241
Private subnet A	10.10.2.0/24	App-A intended/test IP: 10.10.2.10/32; primary IP: 10.10.2.53/24
VPC B	10.20.0.0/16	Local route retained
Private subnet B	10.20.1.0/24	App-B intended/test IP: 10.20.1.10/32; primary IP: 10.20.1.97/24

Note: public IPs can change unless an Elastic IP is used. The previously observed bastion public IP is included for context only; verify the current value before including it in a final report.

6. Network Configuration

6.1 Subnets and route-table associations

Route table	Explicit subnet association	Key routes recorded
ewb-rt-a-public	ewb-a-public (subnet ID to confirm)	10.10.0.0/16 → local; 10.20.0.0/16 → pcx; 0.0.0.0/0 → IGW
ewb-rt-a-private	ewb-a-private (subnet-0558eab73a3d67ba1)	10.10.0.0/16 → local; 10.20.0.0/16 → pcx
ewb-rt-b-private	ewb-b-private (subnet-0b1529b9710d77133)	10.20.0.0/16 → local; 10.10.0.0/16 → pcx

The route table associations for both private subnets were shown as explicit associations. The peer routes were recorded as Active in the AWS route tables. Capture the final route and association tabs for all three tables as evidence.

6.2 Internet gateway

The internet gateway recorded for VPC A is igw-030431e12118d6dad. The public route table has a default route (0.0.0.0/0) to the internet gateway. VPC B does not use an internet gateway in this design.

7. VPC Peering Configuration

The peering connection is named ewb-a-b-peer and has ID pcx-01934796beac9a6c4. The route tables reference this peering ID for traffic between 10.10.0.0/16 and 10.20.0.0/16. Before submission, attach a screenshot from VPC → Peering connections that visibly confirms its current status is Active and identifies both VPCs.

8. Security Configuration

Security group	Required inbound access (confirm actual rules)	Purpose
ewb-sg-bastion	SSH/TCP 22 from current administrator public IP /32	Restricts direct administration access to the bastion.
ewb-sg-app-a	SSH/TCP 22 from actual bastion private IP 10.10.1.26/32; ICMP Echo Request from 10.20.1.10/32	Allows bastion administration and App-B ping requests when sourced from intended test IP.
ewb-sg-app-b	SSH/TCP 22 from actual bastion private IP 10.10.1.26/32; ICMP Echo Request from 10.10.2.10/32	Allows bastion administration and App-A ping requests when sourced from intended test IP.

The lab uses default allow-all outbound security-group rules and default allow-all network ACL rules. Recorded ACLs: App-A subnet acl-0c2be1c848b953608 and App-B subnet acl-035c8991c4f760e19; both were observed allowing all inbound and outbound traffic. Confirm these values before final submission.

Security note: the intended /32 ICMP rules match the explicit test source IPs. The primary addresses are different, so ping without the -I option may use a source address not covered by those rules. Do not broaden access to 0.0.0.0/0 as a permanent fix.

9. SSH Administration Through the Bastion

SSH ProxyJump was configured from the Kali Linux client to access private instances through the bastion. The successful connection details should be evidenced with terminal screenshots. This report does not claim SSH validation as complete until the corresponding screenshots are added.

Example client aliases (do not include private-key contents):

```
Host ewb-bastion
HostName <current-bastion-public-ip>
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem

Host ewb-app-a
HostName 10.10.2.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
ProxyJump ewb-bastion

Host ewb-app-b
HostName 10.20.1.10
User ec2-user
IdentityFile ~/.ssh/ewb-lab.pem
ProxyJump ewb-bastion
```

10. Connectivity Testing and Results

10.1 Forward test: App-A → App-B

Command:

```
ping -I 10.10.2.10 -c 4 -W 2 10.20.1.10
```

Observed result: 4 packets transmitted, 4 received, 0% packet loss. Average round-trip time was approximately 0.177 ms.

10.2 Reverse test: App-B → App-A

Command:

```
ping -I 10.20.1.10 -c 4 -W 2 10.10.2.10
```

Observed result: 4 packets transmitted, 4 received, 0% packet loss. Average round-trip time was approximately 0.167 ms.

Test	Status based on current evidence	Evidence filename to add
App-A → App-B ping	PASS — user-provided terminal output	09-app-a-to-app-b-ping.png
App-B → App-A ping	PASS — user-provided terminal output	10-app-b-to-app-a-ping.png
SSH through bastion to App-A	Needs screenshot / confirmation	11-ssh-through-bastion-app-a.png
SSH through bastion to App-B	Needs screenshot / confirmation	12-ssh-through-bastion-app-b.png
Controlled ICMP failure	Not yet evidenced	13-controlled-icmp-failure.png
ICMP recovery after restoring rule	Not yet evidenced	14-icmp-recovery.png

10.3 Initial failure and controlled failure/recovery

The initial connectivity troubleshooting included failed ping attempts when the default primary source IP was used or when the source did not match the narrow ICMP rule. Explicitly selecting the intended secondary IP address resolved the tested path. This is distinct from the workbook's required controlled failure exercise. For that exercise, temporarily remove the App-B inbound ICMP Echo Request rule, capture the failed App-A ping, restore the rule, and capture the

successful ping again. Only mark this step complete after the screenshots are taken.

11. Traffic Flow: Forward and Return Paths

Forward path (App-A to App-B): App-A sends an ICMP Echo Request from 10.10.2.10 to 10.20.1.10. The subnet's associated route table in VPC A matches the destination against 10.20.0.0/16 and sends the traffic to the VPC peering connection. VPC B receives the packet if its network and security controls permit it.

Return path (App-B to App-A): App-B sends the ICMP Echo Reply back to 10.10.2.10. VPC B's route table matches 10.10.0.0/16 and sends the reply through the peering connection. Security groups are stateful, so response traffic for permitted flows is automatically allowed. The successful ping outputs confirm that the forward and return paths worked for these tests.

12. Cleanup and Cost Control

Cleanup is a required project deliverable, but completion has not been confirmed in the information available for this report. Save all evidence before deleting resources. Follow the EWB workbook's cleanup order: terminate the three lab EC2 instances, check/delete leftover lab EBS volumes or snapshots, release any Elastic IP created for the lab, remove the peering routes and peering connection, delete lab security groups after network interfaces are gone, then remove lab subnets, custom route tables, internet gateway and VPCs. Finally check EC2, VPC, public IPv4 allocations and Billing for remaining chargeable resources.

Cleanup evidence to add: 15-cleanup-ec2.png, 16-cleanup-vpc-resources.png, 17-cleanup-billing-check.png. If you intentionally retain resources, document exactly what remains and why; do not claim full cleanup.

13. Lessons Learned

This project demonstrates that a VPC peering connection alone is not enough to establish communication. The VPC CIDRs must not overlap, routes must exist on both sides, subnet route-table associations must be correct, and security groups must allow the intended traffic. The tests also highlighted the importance of checking the actual source IP used by Linux: an instance with a secondary /32 address may still use its primary address for ordinary outbound traffic unless the source is explicitly selected. Narrow security rules should match the actual traffic design.

14. Screenshot Register — Required GitHub Evidence

Use the exact names below, or keep your own names consistently. Capture genuine AWS Console or terminal output; do not create mock screenshots or edit test results. Place these files in the repository's screenshots/ directory and link them from the README.

Filename	What the screenshot must show	Status
01-vpc-a-created.png	VPC A name, ID and CIDR	Capture / add
02-vpc-b-created.png	VPC B name, ID and CIDR	Capture / add
03-subnets-created.png	Three subnets and their CIDRs	Capture / add
04-internet-gateway.png	IGW attachment and public default route	Capture / add
05-route-tables.png	All three route lists and subnet associations	Capture / add
06-ec2-instances.png	Instance IDs, private IPs; App-A/B have no public IP	Capture / add
07-vpc-peering-active.png	Peering ID, Active status, both VPC IDs	Capture / add
08-security-groups.png	Inbound rules for all three security groups	Capture / add
09-app-a-to-app-b-ping.png	App-A command and 4/4 reply output	Output supplied; save screenshot
10-app-b-to-app-a-ping.png	App-B command and 4/4 reply output	Output supplied; save screenshot

Filename	What the screenshot must show	Status
11-ssh-through-bastion-app-a.png	Successful SSH session to App-A via bastion	Verify / capture
12-ssh-through-bastion-app-b.png	Successful SSH session to App-B via bastion	Verify / capture
13-controlled-icmp-failure.png	Ping failure after temporary ICMP rule removal	Pending
14-icmp-recovery.png	Ping success after rule restoration	Pending
15-cleanup-ec2.png	EC2 lab resources terminated / absent	Pending
16-cleanup-vpc-resources.png	Lab networking resources removed / absent	Pending
17-cleanup-billing-check.png	Billing/resource review evidence, if available	Pending

Appendix A — GitHub Repository Structure

Use a professional, readable structure. Keep the final report in documentation/ and all real screenshots in screenshots/.

```
aws-vpc-peering-project/
■■■ README.md
■■■ documentation/
■ ■■■ StanyJoseph_EWB_VPC_Peering.pdf
■ ■■■ architecture-diagram.png
■ ■■■ resource-inventory.md
■■■ screenshots/
■ ■■■ 01-vpc-a-created.png
■ ■■■ 02-vpc-b-created.png
■ ■■■ 03-subnets-created.png
■ ■■■ 04-internet-gateway.png
■ ■■■ 05-route-tables.png
■ ■■■ 06-ec2-instances.png
■ ■■■ 07-vpc-peering-active.png
■ ■■■ 08-security-groups.png
■ ■■■ 09-app-a-to-app-b-ping.png
■ ■■■ 10-app-b-to-app-a-ping.png
■ ■■■ 11-ssh-through-bastion-app-a.png
■ ■■■ 12-ssh-through-bastion-app-b.png
■ ■■■ 13-controlled-icmp-failure.png
■ ■■■ 14-icmp-recovery.png
■ ■■■ cleanup evidence files
■■■ learning/
■■■ lessons-learned.md
```

Appendix B — Final Submission Checklist

Item	Final check
Cover page	Enter batch and actual completion date.
Resource inventory	Confirm missing IDs and actual current values in AWS Console.
Architecture	Diagram embedded and saved separately as architecture-diagram.png.
Screenshot evidence	All named screenshots exist, are readable, and show genuine results.
Testing	Only claim SSH, controlled failure/recovery and cleanup if verified.
Security	No .pem files, passwords, access keys, tokens, or unredacted sensitive details.
PDF	Save this report as StanyJoseph_EWB_VPC_Peering.pdf in documentation/.
Repository	Open the GitHub link in a signed-out/private browser window if public, or verify trainer access if private.

Prepared for the EWB Courses AWS VPC Peering project. This report documents the known environment and test results; any item marked pending must be completed or updated before submission.